Robô de cargas automáticas do Vecton — resumo para a TI

Data: 16/10/2026 (atualizado com a carga do DRE) · **Responsável pelo projeto:** Ricardo Guimarães (Controladoria)
Código: repositório GitHub `AppsMarcher/Vecton` (branch `main`), pasta `robo-cargas/` · **No servidor:** `C:\robo-cargas`

1. O que é e para que serve

Antes, a carga de vendas do Vecton era manual: atualizava-se a planilha `Painel_Vecton.xlsm` (que roda uma query no Protheus), e o arquivo era carregado na tela do Vecton.
O **robô** faz isso sozinho: lê o Protheus (SQL Server `MARCHER-BD`, base `protheus_producao`), calcula o território de cada venda e grava no Vecton (Supabase), **duas vezes ao dia (12:00 e 23:30)**, sempre para o **mês corrente**.

Cada execução apaga o mês corrente no Vecton e regrava tudo (carga completa). Meses anteriores e futuros continuam sendo carregados à mão, como antes.

2. Como funciona

```
Agendador do Windows (a cada 5 min)  →  robô (Node.js)
   1. entra no Vecton (usuário robo.vecton@marcher.com.br)
   2. lê a configuração (ligada? horários? janela de meses) no próprio Vecton
      e verifica se há pedido "Carregar agora" feito pela tela do Vecton
   3. se há um horário vencido, consulta o Protheus (somente leitura)
   4. resolve o território (tabela Parâmetros → Comercial → Municípios)
   5. cria o lote, o Vecton valida, e o robô aplica
   6. grava o resultado em carga_automatica_log
   7. se der certo: aviso "Carga de Vendas aplicada" (como na carga manual); se der erro: nada é
       aplicado e os administradores recebem e-mail
```

O robô **só lê** do Protheus. Horário, janela de meses e liga/desliga são alterados no Vecton (tabela `carga_automatica_config`), sem mexer no servidor.

Botão "Carregar agora": na tela do Vecton (Parâmetros → Carga de Realizado → seção *Automatizadas*) um administrador pode pedir uma carga fora do horário. O navegador apenas registra o pedido no banco; é o robô, no servidor, que o executa na próxima rodada da tarefa (por isso a tarefa deve rodar a cada 5 minutos). Nenhuma porta do servidor é aberta para a internet.

Cargas: vendas (12:00 e 23:30) e DRE (10:00, 15:00 e 19:00), cada uma com seus horários.

Proteções: extração vazia ou queda de mais de 30% nas linhas em relação à última carga → não aplica e avisa. Qualquer linha reprovada na validação (ex.: conta ou cliente não cadastrado) → não aplica nada e avisa por e-mail, com as linhas agrupadas por motivo. O robô só registra "aplicada" depois de conferir que o lote inteiro está no razão do Vecton.

3. Situação da implantação (16/10/2026)

Item	Situação
Código do robô e testes unitários (26 testes)	Pronto, no GitHub
Banco do Vecton (migrations 259 a 264): configuração, log, mapa de municípios, avisos de erro e de carga concluída, pedidos "Carregar agora" e carga do DRE	Aplicado
Botão "Carregar agora" (migration 263, robô e tela)	**Validado em produção** (16/10): pedido atendido pelo robô em até 5 min, card atualizado sozinho, aviso no sino e por e-mail

Item	Situação
Tela Parâmetros → Comercial → Municípios	Publicada
Usuário `robo.vecton@marcher.com.br` no Vecton	Criado, login validado
Login SQL de leitura `usr_robo_vecton` no `MARCHER-BD`	Criado, validado a partir do servidor
Robô instalado no servidor (`C:\robo-cargas`, Node, `npm install`, `config.json`)	**Concluído**
Simulação (`--dry-run`) a partir do servidor	**Validada**
Carga real a partir do servidor	**Validada**: mês 2026-10, 166 linhas (FAT 51, CART 115), 0 sem território
Carga de vendas ligada, horários `12:00` e `23:30`	Configurado no Vecton
Carga do DRE: ligada, horários próprios `10:00`, `15:00` e `19:00`	Configurado no Vecton
Pasta de cópia dos CSVs	Configurada para `C:\robo-cargas\logs`
Tarefa agendada no Windows (`Vecton - Robo de Cargas`, a cada 5 min)	**Em operação**: ciclo programado das 23:30 e pedidos do botão atendidos
Aviso de carga aplicada (sino e e-mail), nas cargas manual, programada e pelo botão	**Validado**
Carga automática do **DRE** (razão contábil `CT2010`, mesmo esquema das vendas)	**Validada**: simulação bateu com o Excel (596 lançamentos, saldo 465.317,39) e a carga real foi aplicada; DRE/OPEX conferidos
Relatórios (DRE/OPEX) atualizando após carga do robô	**Corrigido** (descarte do cache do navegador); exige recarregar o Vecton uma vez
Robô com conferência do razão (versão atual)	Publicado no GitHub; **a atualizar no servidor** (ver 4.1.2)
Aviso por e-mail em caso de erro	Implementado; ainda não exercitado (só dispara numa falha real)

4. O que falta — pendências da TI e referência de instalação

4.1 Tarefa agendada (concluído; referência para recriar)

O script `instalar-tarefa.ps1` cria a tarefa `Vecton - Robo de Cargas`, que dispara a cada 5 minutos (parâmetro `-IntervaloMin`); o robô decide sozinho se há horário vencido (12:00 e 23:30). Hoje ela roda com a conta `marcher.local\ti.marcher`.

1. Em PowerShell **como administrador**, em `C:\robo-cargas`:

```
.\instalar-tarefa.ps1 -Conta "marcher.local\ti.marcher" -Escopo Machine
```

   O script pede: a senha da conta do Windows, a senha do Vecton e a do SQL Server. O `-Escopo Machine` grava as senhas como variáveis do servidor.

   Se aparecer `not digitally signed`: `Unblock-File .\instalar-tarefa.ps1` e, se preciso, `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` (vale só para a janela).
2. Conferir no Agendador de Tarefas que a tarefa existe e o "Resultado da última execução" é `0x0`.
3. Conferir o log (seção 6): deve aparecer uma linha `ok` por horário programado (12:00 e 23:30).

4.1.1 Atualizar o robô no servidor (feito em 16/10; repetir a cada nova versão)

1. Atualizar os arquivos da pasta `C:\robo-cargas` com a versão atual do GitHub (principalmente `src\runner.js` e `instalar-tarefa.ps1`).
2. Recriar a tarefa (o script substitui a existente) para que ela passe a rodar a cada 5 minutos:

```
.\instalar-tarefa.ps1 -Conta "marcher.local\ti.marcher" -Escopo Machine
```

4.1.2 Atualizar o robô no servidor para a versão com conferência do razão (pendente)

O DRE já está ligado e validado. A versão atual do robô acrescenta a conferência do razão antes de registrar "aplicada". Sem atualizar, as cargas **continuam funcionando**, só sem essa proteção.

1. Copiar do GitHub (pasta `robo-cargas`) para `C:\robo-cargas`, **os quatro juntos**: `src\runner.js`, `src\supabase.js`, `src\jobs\vendas.js` e `src\jobs\dre.js`. O `runner.js` novo exige que as duas cargas informem o seu razão, por isso nenhum deles pode ficar para trás.
2. Se ainda não existirem: `src\erros.js` e `sql\dre_mes.sql` (usados pela carga do DRE).
3. Não é preciso recriar a tarefa, rodar `npm install` nem mexer em senhas ou `config.json`.
4. Conferir: `node src/index.js dre --dry-run` deve mostrar o resumo normal, sem `Cannot find module`.

O login `usr_robo_vecton` precisa ler `CT2010` (no ambiente atual já lê todas as tabelas).

4.2 Conta definitiva para a tarefa

A tarefa roda com `marcher.local\ti.marcher`. Se a senha dessa conta for trocada, a tarefa para até ser recriada. O ideal é uma **conta de serviço** do domínio, de senha estável, com o direito "Fazer logon como tarefa em lote" e acesso a `C:\robo-cargas`. Para trocar, rodar o mesmo script com o novo `-Conta` (ele substitui a tarefa existente).

4.3 Pasta de cópia dos arquivos (opcional)

O robô grava `Vendas_AAAA-MM.csv` e `DRE_AAAA-MM.csv` em `C:\robo-cargas\logs` (sobrescrito a cada execução). Se a Controladoria quiser a cópia em rede (ex.: `3 - CARGAS ENVIADAS`), é preciso o **caminho UNC** (`\\servidor\...`) e permissão de escrita para a conta da tarefa, pois ela não enxerga unidades mapeadas (`O:`). Se a cópia falhar, a carga **não falha**: o aviso fica no log.

4.4 Referência: instalação do robô em um servidor novo

1. Servidor Windows ligado 24x7 com acesso a `MARCHER-BD` (SQL Server) e à internet (`*.supabase.co`, HTTPS 443); Node.js 20 ou superior.
2. Copiar a pasta `robo-cargas` do GitHub para um disco **local** (ex.: `C:\robo-cargas`) e rodar `npm install`.
3. Copiar `config.example.json` para `config.json` e ajustar o bloco do banco para o login SQL (o arquivo **não tem senhas**):

```
"protheus": { "server": "MARCHER-BD", "database": "protheus_producao",
              "auth": "sql", "user": "usr_robo_vecton", "passwordEnv": "PROTHEUS_SENHA" }
```

4. Testar sem gravar (senhas só para a janela, com aspas simples):

```
$env:VECTON_ROBO_SENHA = 'senha-do-vecton'
$env:PROTHEUS_SENHA = 'senha-do-banco'
node src/index.js vendas --dry-run
```

   Esperado: `[SIMULACAO] vendas 2026-10: ... linhas (FAT ..., CART ...); sem territorio: 0; sem tipo: 0` e "nada foi gravado no Vecton".
5. Agendar a tarefa (seção 4.1).

5. Segurança

- O robô **só lê** do Protheus: o usuário `usr_robo_vecton` precisa apenas de `SELECT` nas tabelas `SD2010, SD1010, SB1010, SF2010, SA1010, SC5010, SC6010, SF4010, SYD010` (vendas) e `CT2010` (DRE).

- Senhas **nunca** em arquivo nem no repositório: ficam em variáveis de ambiente (`VECTON_ROBO_SENHA` e `PROTHEUS_SENHA`). O `config.json` está fora do Git.
- O usuário do robô no Vecton tem perfil Administrador (será usado para outras cargas).

6. Como acompanhar e operar

- **Log de execuções** (SQL Editor do Supabase):

```
select started_at, status, reference_month, total_rows, mensagem
from public.carga_automatica_log where job = 'vendas' order by started_at desc limit 10;
```

  Status: `ok`, `erro`, `bloqueada`.
- **Pausar:** `update public.carga_automatica_config set enabled = false where job = 'vendas';` (religar com `true`).
- **Alterar horários (cada carga tem os seus):** `update public.carga_automatica_config set horarios = array['12:00','23:30'] where job = 'vendas';` e `... array['10:00','15:00','19:00'] where job = 'dre';` (não usar `00:00`).
- **Rodar agora, à mão:** `node src/index.js vendas` na pasta do robô (apaga e regrava o mês corrente e dispara os avisos de carga aplicada).
- **Quando der erro:** o e-mail chega aos administradores (lista em Parâmetros → Notificações → "Carga automática com erro"). Causa comum: cliente ou produto não cadastrado; basta cadastrar e a próxima execução carrega sozinha.
- Documentação técnica completa: `robo-cargas/README.md`.

7. Pontos em aberto

1. Atualizar o robô no servidor para a versão com conferência do razão (seção 4.1.2).
2. Conta de serviço definitiva para a tarefa (seção 4.2).
3. Pasta de rede (UNC) para a cópia dos CSVs, se desejada (seção 4.3).